

SLIDE 1 — Capstone Project: Using Arrays

Definition:

A **capstone project** combines what we learned about arrays to solve a simple real-world problem.

Simple idea

Input → **Store** → **Process** → **Display**

Example: Student Marks

```
int marks[5] = {80, 75, 90, 85, 95};
```

We can use the array to:

- Display marks
- Calculate total
- Find average
- Find highest mark
- Find lowest mark

Explanation:

Student Marks



Array



Calculate Results



Display Output

 **Remember: Project = Combine array concepts!**

SLIDE 2 — Mini Capstone Example

Student Marks Analyzer

```
int marks[5] = {80, 75, 90, 85, 95};  
int sum = 0;
```

```
for (int i = 0; i < 5; i++) {  
    sum += marks[i];  
}
```

```
printf("Total = %d", sum);
```

Output:

Total = 425

What did we use?

- Array → Store marks
 - Loop → Visit elements
 - `sum` → Calculate total
-

SLIDE 3 — Array Limitations

Arrays are useful, but they have some limitations.

1. Fixed Size

```
int numbers[5];
```

The size is fixed when the array is created.

2. Same Data Type

```
int numbers[5];
```

It stores integer values.

3. Out-of-Bounds Risk

```
numbers[5]; // ❌ Invalid
```

For size 5, valid indexes are **0–4**.

4. Insertion & Deletion

Adding or removing elements can require shifting other elements.

 **Remember: Array = Simple but Limited**

SLIDE 4 — Choosing the Right Array

Before creating an array, think about:

How many elements?

```
int marks[50];
```

Choose a suitable size.

What type of data?

```
int ages[10];
```

```
float prices[10];
```

```
char name[20];
```

What operations are needed?

Store → Access → Search → Calculate

Safety Check

Always stay within the valid range:

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]);  
}
```

Basic Pattern:

PLAN → CREATE → PROCESS → CHECK

SLIDE 5 — RECAP

Arrays: From Basics to Project! 

Flow

CREATE ARRAY



STORE DATA



USE LOOPS



PROCESS DATA



DISPLAY RESULT

5 Things to Remember

1. Arrays store multiple values
2. Loops help process array elements
3. Functions can process arrays
4. Array size is generally fixed
5. Always avoid out-of-bounds access

Final Example

```
int marks[3] = {80, 90, 70};
```

```
int sum = 0;
```

```
for (int i = 0; i < 3; i++)  
    sum += marks[i];
```

```
printf("Total = %d", sum);
```

Output:

Total = 240

Memory Trick

ARRAY → STORE 

LOOP → PROCESS 

FUNCTION → REUSE 

QUESTIONS

1. What is the purpose of a capstone project?

- A) To learn only one concept
- B) To combine learned concepts to solve a problem
- C) To delete arrays
- D) To avoid using loops

 **Hint:** A project combines multiple concepts.

 **Glossary: Capstone** → A final project that combines learned concepts.

 **Answer: B**

2. What is a major limitation of a normal array?

- A) It cannot store values
- B) Its size is generally fixed
- C) It cannot be accessed
- D) It cannot use loops

 **Hint:** Think about what happens after the array is created.

 **Glossary: Fixed size** → A size that cannot normally be changed after creation.

 **Answer: B**

3. What is the last valid index?

`int numbers[5];`

- A) 3
- B) 4
- C) 5
- D) 6

 **Hint:** Last index = size - 1.

 **Glossary:** Valid index → An index within the allowed array range.

 **Answer:** B) 4

4. Fill in the blank

Complete the code to calculate the total:

```
int numbers[3] = {10, 20, 30};  
int sum = 0;
```

```
for (int i = 0; i < 3; i++) {  
    sum += _____;  
}
```

 **Hint:** Access the current element using the loop index.

 **Glossary:** Current element → The element at the current index.

 **Answer:** `numbers[i]`

5. What should you avoid when working with arrays?

- A) Using loops
- B) Using indexes
- C) Out-of-bounds access
- D) Calculating sums

 **Hint:** Accessing outside the valid range is unsafe.

 **Glossary:** Out-of-bounds → Accessing an invalid array index.

 **Answer:** C

6. Why are loops useful in a capstone array project?

- A) They process multiple elements efficiently
- B) They remove the array

- C) They change the data type
- D) They stop all calculations

 **Hint:** A loop can visit every element.

 **Glossary: Process** → Perform an operation on data.

 **Answer: A**

7. Arrange the steps in the correct order:

- A. Display result
- B. Store data
- C. Process data
- D. Create array

 **Hint:** Create → Store → Process → Display.

 **Glossary: Display** → Show the result to the user.

 **Answer: D → B → C → A**